

Verifica automatica di programmi

Riccardo Torre

17 aprile 2026

1 BubbleSort

Definizione di Partitioned

$$\overbrace{\text{Partitioned}(a, l_1, u_1, l_2, u_2)}^P \equiv \forall i \forall j: l_1 \leq i \leq u_1 < l_2 \leq j \leq u_2 \implies a[i] \leq a[j]$$

Definizione di Sorted

$$\overbrace{\text{Sorted}(a, l, u)}^S \equiv \forall i \forall j: l \leq i \leq j \leq u \implies a[i] \leq a[j]$$

```
1  pre:  ⊤
2  post: sorted(rv, 0, |rv| - 1)
3
4  int[] BubbleSort(int[] a0){
5
6      for @L1: -1 ≤ i < |a| ∧  $\overbrace{P(a, 0, i, i+1, |a|-1)}^A \wedge \overbrace{S(a, i, |a|-1)}^B$ 
7      (int i := |a| - 1; i > 0; i := i+1){
8          for @L2: 1 < i < |a| ∧ 0 ≤ j ≤ i ∧ A ∧ B ∧ P(a, 0, j-1, j, j)
9          (int j := 0; j < i; j := j+1)
10             if(a[j] > a[j+1]){
11                 int t := a[j];
12                 a[j] := a[j+1];
13                 a[j+1] := t;
14             }
15     }
16     return a;
}
```

1.1 Cammini elementari di BubbleSort

Inizio ciclo esterno $P(a, \overbrace{0}^{l_1}, \overbrace{|a|-1}^{u_1}, \overbrace{|a|-1}^{l_2}, \overbrace{|a|-1}^{u_2})$ è vera a vuoto perché $\nexists j$ che verifica. Applichiamo la definizione di **Partitioned**.

$$\overbrace{\text{Partitioned}(a, l_1, u_1, l_2, u_2)}^P \equiv \forall i \forall j: l_1 \leq i \leq u_1 < l_2 \leq j \leq u_2 \implies a[i] \leq a[j]$$

Sostituendo si ottiene $P(a_0, 0, |a_0| - 1, |a_0|, |a_0| - 1) \equiv \forall i \forall j: 0 \leq i \leq |a_0| - 1 < |a_0| \leq j \leq |a_0| - 1 \implies a[i] \leq a[j]$. Ma $\nexists j: |a_0| \leq j \leq |a_0| - 1$ perché $|a_0| - 1 \not\leq |a_0|$.

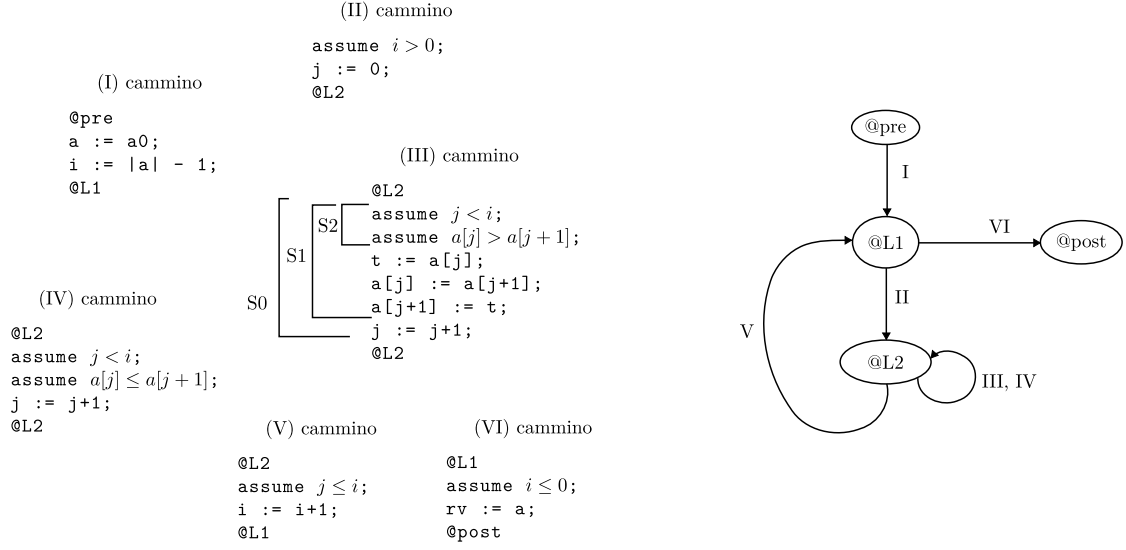


Figura 1: cammini minimi e grafo di BubbleSort

Inizio ciclo interno $P(a, 0, -1, 0, 0)$ è vera a vuoto poiché tra 0 e -1 non esiste nulla ($\nexists i \in [0, -1]$).

1.2 Condizioni di verifica per i cammini del grafo

Weakest precondition del cammino (I) $wp(-1 \leq i < |a| \wedge A \wedge B, a := a0; i := |a| - 1;) \equiv (-1 \leq |a| - 1 < |a| \wedge P(a, 0, |a| - 1, |a|, |a| - 1) \wedge S(a, |a| - 1, |a| - 1), a := a0) \equiv (-1 \leq |a0| - 1 < |a0| \wedge P(a0, 0, |a0| - 1, |a0|, |a0| - 1) \wedge S(a0, |a0| - 1, |a0| - 1))$.

Condizione di verifica per il cammino (I) $T \implies (-1 \leq |a0| - 1 < |a0| \wedge P(a0, 0, |a0| - 1, |a0|, |a0| - 1) \wedge S(a0, |a0| - 1, |a0| - 1))$ è vera perché:

- P è vera;
- S è vera;
- $-1 \leq |a0| - 1 < |a0|$ vera anche per $|a0| = 0$.

Weakest precondition del cammino (II) $wp(1 \leq i < |a| \wedge 0 \leq j \leq i \wedge A \wedge B \wedge P(a, 0, j - 1, j, j), \text{assume } i > 0; j := 0;) \equiv i > 0 \implies 1 \leq i < |a| \wedge 0 \leq 0 \leq i \wedge P(a, 0, i, i + 1, |a| - 1) \wedge S(a, i, |a| - 1) \wedge P(a, 0, -1, 0, 0)$.

Condizione di verifica per il cammino (II) $[-1 \leq i < |a| \wedge P(a, 0, i, i + 1, |a| - 1) \wedge S(a, i, |a| - 1) \wedge i > 0] \implies [1 \leq i < |a| \wedge 0 \leq i \wedge P(a, 0, i, i + 1, |a| - 1) \wedge S(a, i, |a| - 1) \wedge P(a, 0, -1, 0, 0)]$. L'implicazione è vera.

Weakest precondition del cammino (III) $wp(1 \leq i < |a| \wedge 0 \leq j \leq i \wedge P(a, 0, i, i + 1, |a| - 1) \wedge S(a, i, |a| - 1) \wedge P(a, 0, j - 1, j, j), S_0) = wp(1 \leq i \leq |a| \wedge 0 \leq j + 1 \leq i \wedge P(a, 0, i, i + 1, |a| - 1) \wedge S(a, i, |a| - 1) \wedge P(a, 0, j, j + 1, j + 1), S_1) = wp(1 \leq i < |b| \wedge 0 \leq j + 1 \leq i \wedge P(b, 0, i, i + 1, |b| - 1) \wedge S(b, i, |b| - 1) \wedge P(b, 0, j, j + 1, j + 1), S_2)$

$t := a[j]$ diventa $t \leftarrow \text{select}(a, j)$
 $a[j] := a[j+1];$ diventa $a \leftarrow \text{store}(a, j, \text{select}(a, j+1))$
 $a[j+1] := t;$ diventa $a \leftarrow \text{store}(a, j+1, t)$
 combinando la seconda e la terza formula si ottiene $a \leftarrow \text{store}(\text{store}(a, j, \text{select}(a, j+1)), j+1, t)$ e sostituendo t con la prima formula si ottiene $a \leftarrow \text{store}(\text{store}(a, j, \text{select}(a, j+1)), j+1, \text{select}(a, j)) \equiv b$ dove b è una **variabile temporanea** di tipo array.

dunque la weakest precondition per il cammino (III) risulta: $[j < i \wedge \text{select}(a, j) > \text{select}(a, j+1)] \implies [1 \leq i < |b| \wedge 0 \leq j+1 \leq i \wedge P(b, 0, i, i+1, |b|-1) \wedge S(b, i, |b|-1) \wedge P(b, 0, j, j+1, j+1)]$

Condizione di verifica per il cammino (III) $[1 \leq i < |a| \wedge 0 \leq j \leq i \wedge P(a, 0, i, i+1, |a|-1) \wedge S(a, i, |a|-1) \wedge P(a, 0, j-1, j, j) \wedge j < i \wedge \text{select}(a, j) > \text{select}(a, j+1)] \implies [1 \leq i < |b| \wedge 0 \leq j+1 \leq i \wedge P(b, 0, i, i+1, |b|-1) \wedge S(b, i, |b|-1) \wedge P(b, 0, j, j+1, j+1)]$.

L'implicazione è vera siccome b è il risultato di due **store** e allora $|b| = |a|$ ed è facile vedere che vengono le altre parti dell'implicazione.

Weakest precondition del cammino (IV) $wp(1 \leq i < |a| \wedge 0 \leq j \leq i \wedge A \wedge B \wedge P(a, 0, j-1, j, j, \text{assume } j < i; \text{assume } a[j] \leq a[j+1]; j := j+1;)) \equiv [j < i \wedge \text{select}(a, j) \leq \text{select}(a, j+1)] \implies [1 \leq i < |a| \wedge 0 \leq j+1 \leq i \wedge A \wedge B \wedge P(a, 0, j, j+1, j+1)]$.

Condizione di verifica per il cammino (IV) $[1 \leq i < |a| \wedge 0 \leq j \leq i \wedge A \wedge B \wedge P(a, 0, j-1, j, j) \wedge j < i \wedge \text{select}(a, j) \leq \text{select}(a, j+1)] \implies [1 \leq i < |a| \wedge 0 \leq j+1 \leq i \wedge A \wedge B \wedge P(a, 0, j, j+1, j+1)]$. L'implicazione è vera.